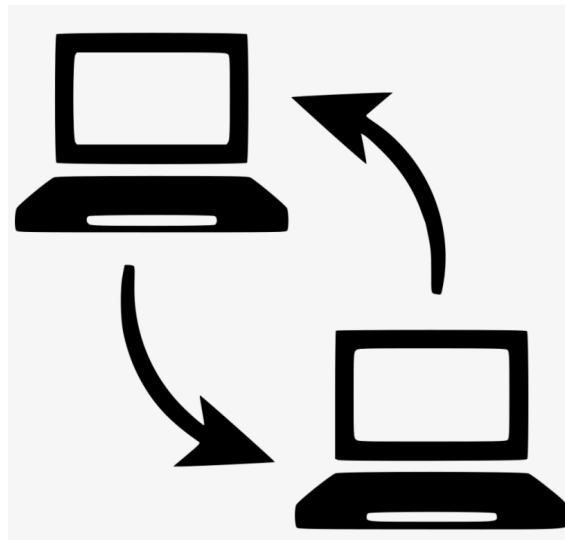


Project report

High Performance Computing for Weather and Climate, ETH Zürich

Communication strategies for halo-updates

**Authors**

Nico Jungck, 21-923-982

Ida Vetsch, 20-934-031

Till Wenke, 25-931-163

Advisor

Max Harvey

Project repositoryhttps://github.com/tillwenke/hpc_mpi

September 15, 2026

Abstract

Halo exchange can limit the parallel performance of stencil computations in weather and climate models. This report compares blocking and non-blocking communication, overlap of communication with computation, and explicit corner exchange for a fourth-order diffusion stencil. The implementations are validated against a serial reference and assessed through strong- and weak-scaling experiments on the Santis supercomputer with up to 96 MPI ranks. Non-blocking communication performs best among the communication-ordering strategies, while blocking schemes with long sequential dependencies scale poorly. Overlapping communication with computation reduces waiting time but does not consistently improve total runtime in the tested configurations. Explicit corner exchange reduces halo-exchange time by up to approximately 15% at high rank counts. Overall, the results highlight the benefit of reducing sequential waiting, while showing that additional overhead can offset the gains from communication optimizations.

Contents

1	Introduction	3
2	Methods	3
2.1	Blocking and Non-Blocking Communication	3
2.2	Overlapping Computation and Communication	4
2.3	Explicit Corner Exchange	5
2.4	Validation	6
2.5	Experimental Setup	7
3	Results	7
3.1	Blocking and Non-Blocking Communication	7
3.1.1	Weak Scaling	8
3.1.2	Strong Scaling	9
3.2	Overlapping Computation and Communication	9
3.3	Explicit Corner Exchange	11
4	Discussion and Conclusion	12
5	Outlook	13
A	Detailed Corner-Exchange Timings	14
A.1	Packing and Unpacking	14
A.2	Send and Receive Initiation	14
A.3	Waiting for Communication	14
B	Statement of AI use	16

1 Introduction

Stencil-based weather and climate code is highly parallelizable, offering potential for scaling on supercomputers. This introduces the new problem of communication between processing units that don't have shared memory access. Halo exchange is the dominant communication pattern in stencil computations on a grid. At every timestep, each rank must send the data on the boundary of its subgrid to, and receive respective boundaries from, its neighbor ranks on the grid before computation can proceed. How this communication is implemented is a technical choice that can affect performance.

The task evolves a field of size $n_x \times n_y \times n_z$ for `num_iter` diffusion iterations. For P MPI ranks, the code chooses the closest factor pair $P_x P_y = P$ to form a two-dimensional process grid, partitions n_x across P_x and n_y across P_y , and keeps the full n_z extent on every rank.

This report investigates three largely independent design questions for the halo exchange step of a 4th-order diffusion stencil (`stencil2d`):

1. **Blocking and non-blocking communication.** Blocking `Send/Recv` calls between neighboring ranks can deadlock. We compare three ways of avoiding this, library-managed ordering (`sendrecv`), a checkerboard two-coloring of the process grid (`sendrecv_evenodd`), and a cycle-breaking scheme that does not require a two-coloring (`broken_cycles`), against a non-blocking reference (`Irecv/Isend`). We are especially interested in whether the explicit communication ordering in `sendrecv_evenodd` and `broken_cycles` behaves differently from the library-managed approaches.
2. **Overlap of computation and communication.** In principle, non-blocking communication allows the interior of the domain (which does not depend on halo data) to be computed while halo messages are in flight. We test whether this pays off in practice.
3. **Explicit corner exchange.** The stencil used here applies the Laplacian operator twice per timestep, which makes diagonal (corner) neighbor data relevant two grid points away. A communication strategy that is simple in the way that each grid cell only has to communicate to its direct neighbors is the sequenced top/bottom-then-left/right exchange. However this comes at the cost of additional synchronization points. We test whether explicitly exchanging corner data with diagonal neighbors improves performance, where communication does not need to happen sequentially.

2 Methods

2.1 Blocking and Non-Blocking Communication

All strategies in this section are implemented in `stencil2d-comm-strats_timer.F90`, selected at runtime via the `--comm_strategy` flag. `stencil2d-comm-strats_timer.F90` is structured such that the sole variable between strategies is the communication mechanism; that is, all methods share identical packing, buffer management, and top/bottom, left/right sequencing.

- **Irecv/Isend** (reference): both the send and the receive are posted non-blocking. This is deadlock-free by construction, since no call blocks waiting for a peer. Note that `MPI.Waitall` is called inside the halo-update routine, before it returns. Hence, there is no overlapping communication and computation.
- **sendrecv**: a single blocking `MPI.Sendrecv` call per direction pair. This is deadlock-free by construction, since the MPI library internally sequences the send and receive.
- **Irecv/Send, Isend/Recv**: receives are posted non-blocking while sends are blocking (or vice versa). Included as intermediate points between the fully blocking and fully non-blocking extremes.
- **sendrecv_evenodd**: plain blocking `Send/Recv`, deadlock-avoided by a checkerboard two-coloring of the process grid: The ranks are placed on a process grid and each rank's color is $(\text{row} + \text{col}) \bmod 2$; color-0 ranks send to neighbors first and then receive, color-1 ranks do the reverse. For this method to work, every pair of adjacent ranks on the process grid must have opposite colors. This is guaranteed for all interior ranks by definition. However, due to the

circular dependence structure, a periodic boundary condition requires that the process grid has an even number of ranks along both axes. This is a necessary and sufficient condition since from graph theory we know that a cycle has a valid two-coloring if and only if it is even.

- **brokencycles**: plain blocking **Send/Recv**, deadlock-avoided by breaking cycles of communication dependencies. The top/bottom and left/right exchanges are each split into two sequential cycles (e.g. “send my top edge to my top neighbor, receive from my bottom neighbor” as one complete cycle). Within each cycle, every rank sends first except the rank at the low end of that axis (`position(2) == 1` for top/bottom, `position(1) == 1` for left/right), which receives first, breaking the cycle and seeding a reverse cycle of receives back through the chain. Unlike `sendrecv_evenodd`, this scheme does not require an even number of ranks along either axis, at the cost of a longer dependency chain: a message between two ranks M steps apart along an axis may need $O(M)$ prior sends/receives elsewhere on that axis to resolve, whereas `sendrecv_evenodd` resolves every neighbor pair independently in $O(1)$ steps. Hence, we expect that **brokencycles** should scale worse than `sendrecv_evenodd` as the number of ranks grows.

We compare the performance of these strategies in Section 3.1, where we omit the hybrid **Irecv/Send**, **Isend/Recv** versions because of difficulty of interpretation.

2.2 Overlapping Computation and Communication

`stencil2d-comm_comp.F90` restructures the timestep loop itself to compute the Laplacian on the interior of the domain, the region that does not depend on halo data, while the non-blocking halo exchange (**Irecv/Isend**) is in flight, and only computes the boundary region after the exchange completes. This strategy overlaps communication with computation.

The implementation stores the current field in `in_field` and uses two auxiliary arrays, `tmp1_field` and `tmp2_field`. The first Laplacian is split into an interior region and boundary regions. At the beginning of each timestep, non-blocking receives are posted for the four neighboring faces. The corresponding send buffers are packed and the non-blocking sends are started. The interior part of the first Laplacian is then computed while the halo messages are in flight. Once the required halo data has arrived, it is unpacked and the remaining boundary regions of the first Laplacian are evaluated. The second Laplacian is applied only after the complete intermediate field is available.

The timestep therefore has the following logical structure:

1. post the non-blocking halo exchange;
2. compute the interior of the first Laplacian;
3. wait for and unpack the halo data;
4. compute the boundary of the first Laplacian;
5. apply the second Laplacian;
6. update the output field and copy it back for the next iteration.

The theoretical speedup limit compared to the sequential implementation is

$$S_{max} = \frac{t_{comm} + t_{compute}}{\max(t_{comm}, t_{compute})} \quad (1)$$

The relevant quantity in practice is the local grid size per rank. Growing the local grid makes computation scale faster than halo communication, so communication shrinks as a share of total runtime; hiding it then buys progressively less, and the speedup approaches one. Shrinking the local grid at fixed global size, e.g. by adding ranks, has the opposite effect: communication becomes proportionally more significant and the potential benefit of overlap grows, up to the point where communication dominates and there is too little computation left to hide it. Adding nodes mainly changes communication latency and network overhead and does not by itself guarantee a speedup. This bound is optimistic for two reasons. First, only the interior computation can overlap with communication; the boundary computation is on the critical path and must wait for the halo data regardless. Second, packing, unpacking, request management, synchronization, and the field update

all add overhead that has no counterpart in the sequential version. Reducing communication wait time is therefore necessary but not sufficient for reducing total runtime.

2.3 Explicit Corner Exchange

The diffusion stencil applies the 5-point Laplacian operator twice per timestep. Because the second application reads neighboring values of the *already-differenced* first-pass field, a point near a subdomain’s edge can depend on data that is diagonally adjacent on the original grid – which is why the halo width is 2 rather than 1. The baseline strategies (Section 2.1) handle this implicitly: the top/bottom exchange is completed and unpacked *before* the left/right exchange is packed, so left/right messages already carry the correct diagonal corner data along for free, without any additional messages. This adds synchronization: the left/right exchange cannot begin until the top/bottom exchange has completed. We assume that this sequential dependency can become a bottleneck because each rank must wait for its top/bottom neighbors before it can start sending to its left/right neighbors. We propose a different explicit corner exchange strategy where ranks that are responsible for diagonally adjacent grid cells communicate with each other to exchange the corners of the halo.

A simple model of the halo update. To understand the possible delay, we split each halo update into four steps: packing data into send buffers, posting the MPI calls, transferring the data, and unpacking the received data. The model makes four simplifying assumptions:

1. Packing, posting, and unpacking require the same total work in every strategy.
2. The same total amount of data is transferred; corner values either travel inside the left/right messages or in separate diagonal messages.
3. A transfer can start only after both ranks have posted the matching send and receive.
4. The slowest rank determines the halo-update time because all ranks need complete halos before continuing.

Under these assumptions, explicit corner exchange can only save time by removing the delay between the top/bottom and left/right phases. It posts all edge and corner messages together, so a fast rank does not have to wait for a neighbor to reach a later communication phase.

We benchmark three explicit-corner strategies. All exchange data with the four edge and four diagonal neighbors in one communication phase, removing the dependency between the top/bottom and left/right exchanges:

- **corners** (`stencil2d-mpi-corners.F90`): pack and post all eight exchanges, complete them with one `MPI.Waitall`, then unpack.
- **corners-pipelined** (`stencil2d-mpi-corners-pipelined.F90`): send-side pipelining. Pre-post the receives, then post each group of sends as soon as it is packed, overlapping early transfers with later packing.
- **corners-waitany** (`stencil2d-mpi-corners-waitany.F90`): receive-side pipelining. Use `MPI.Waitany` to unpack each halo as it arrives, overlapping early unpacking with later transfers.

Figure 1 illustrates the expected timing effect of removing the phase dependency and of the two forms of pipelining.

Figure 10 consists of four horizontal bar charts, each representing a different corner handling strategy. The x-axis for all charts is 'Illustrative halo-update time (arbitrary units)' ranging from 0 to 5. The y-axis lists four ranks: Rank 0, Rank 1, Rank 2, and Rank 3. The legend indicates five operations: pack (blue), post receives and sends (yellow), data transfer (red), matching send not posted (grey), and unpack (teal). A vertical dashed line in each chart marks the 'slowest-rank time'.

- Default: sequential edge exchange:** Shows sequential execution. Rank 0 has a long grey segment (matching send not posted) and a long red segment (data transfer). Rank 1 has a long red segment. Rank 2 has a long red segment. Rank 3 has a long red segment. Annotations indicate that Rank 1's long first transfer delays its second post, and Rank 0 cannot receive yet.
- Explicit corners: one exchange phase:** Shows a single exchange phase. Rank 0 has a long red segment. Rank 1 has a long red segment. Rank 2 has a long red segment. Rank 3 has a long red segment.
- Pipelined corners: post overlaps pack:** Shows pipelined execution. Rank 0 has a long red segment. Rank 1 has a long red segment. Rank 2 has a long red segment. Rank 3 has a long red segment. Annotations indicate that Rank 1's post overlaps Rank 0's pack, and Rank 2's post overlaps Rank 1's pack.
- Waitany corners: unpack overlaps transfer:** Shows waitany execution. Rank 0 has a long red segment. Rank 1 has a long red segment. Rank 2 has a long red segment. Rank 3 has a long red segment. Annotations indicate that Rank 1's unpack overlaps Rank 0's transfer, and Rank 2's unpack overlaps Rank 1's transfer.

Narrow hatched regions show the overlapped fraction; the serial remainder still extends the bar.

We compare these variants with the non-blocking `mpi` (`stencil2d-mpi.F90`) baseline.

All communication strategies were validated for correctness before any performance measurement, by comparing the full output field against a serial reference implementation (`stencil2d-orig.F90`) using `compare_fields_f90.py` at $n_x = n_y = 128$, $n_z = 4$, 1024 iterations, 16 ranks on 4 Santis nodes (4 ranks per node, one GH200 per rank).

6

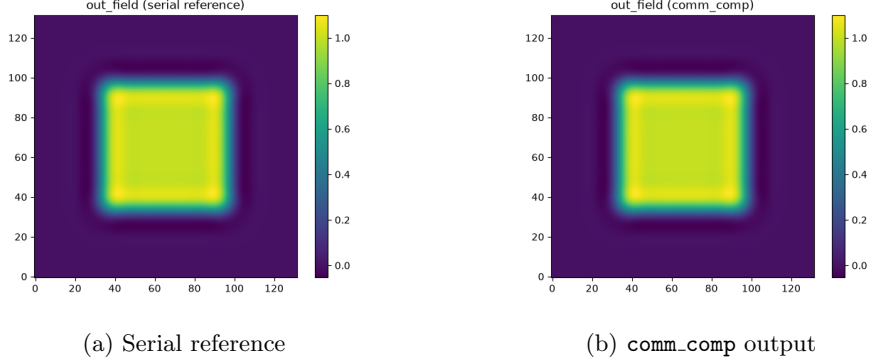


Figure 2: Exemplary output-field heatmaps used for validation.

2.5 Experimental Setup

We study both weak and strong scaling. In both regimes we vary only the number of MPI ranks and the global domain dimensions required by the scaling definition; the numerical method, $n_z = 64$, the iteration count, and the placement policy remain fixed between strategies.

Weak scaling. The domain grows proportionally with the rank count, holding the local subdomain size constant at $128 \times 128 \times 64$ grid points per rank. For each rank count, the launcher chooses the most nearly square two-dimensional process grid and sets the global n_x and n_y accordingly. Weak-scaling efficiency is the runtime at the smallest common rank count divided by the runtime at each larger rank count.

Strong scaling. The domain is fixed at 1024×1024 with $n_z = 64$, while the number of ranks increases. Strong-scaling speedup is measured relative to the smallest common rank count and is compared with ideal linear speedup.

The benchmark sweep runs were performed using `experiment_scaling_all_strategies.sh`. Each Santis node contains four GH200 modules. We place exactly one MPI rank on each GH200, request $\lceil N/4 \rceil$ nodes for N ranks, and use at most four ranks per node. Each rank is assigned one GPU, one CPU core. Runs with up to four ranks therefore communicate only within one node. Beginning at five ranks, the job spans multiple nodes and at least some halo messages cross the inter-node network. Because inter-node transfers are substantially slower than communication between GH200 modules within one node, the measured halo-exchange time is then expected to be dominated by inter-node communication. We consequently expect a performance discontinuity or slowdown at this placement boundary, in addition to the scaling behavior caused by changing the workload per rank.

Each case uses 1024 iterations and is repeated three times. We measure the mean diffusion-loop elapsed time across ranks as the application-level performance metric. To explain differences between strategies, we additionally record the average and maximum communication and computation times across ranks. Here communication time is the time spent in the `update_halo` routine, which is the routine rewritten for the different communication strategies. Where finer analysis is needed, particularly for the corner variants and overlapping computation and communication, the timing is further separated into packing and unpacking, posting sends and receives, and waiting for MPI requests to complete. These component timings distinguish network or synchronization costs from local buffer-management overhead.

3 Results

The commands used to obtain our results and plots can be found in our project’s code repository.

3.1 Blocking and Non-Blocking Communication

This section compares `sendrecv`, `sendrecv_evenodd`, and `brokenecycles` against the `irecvsend` reference, isolating the effect of how a deadlock-free ordering of communication is achieved, inde-

pendent of the overlap and corner-handling questions addressed in Sections 3.2–3.3.

3.1.1 Weak Scaling

In a weak-scaling experiment, the amount of work given to each rank is held fixed, and the total problem size grows only because more ranks are added. If communication were free, total runtime should therefore stay perfectly flat as rank count increases. Figure 3 confirms this is exactly what happens. The orange bars represent the average time spent on computation in the experiments and stay essentially constant across all rank counts and strategies. The small variations are explained by run-to-run noise. This confirms that any difference in runtime between strategies is coming from communication.

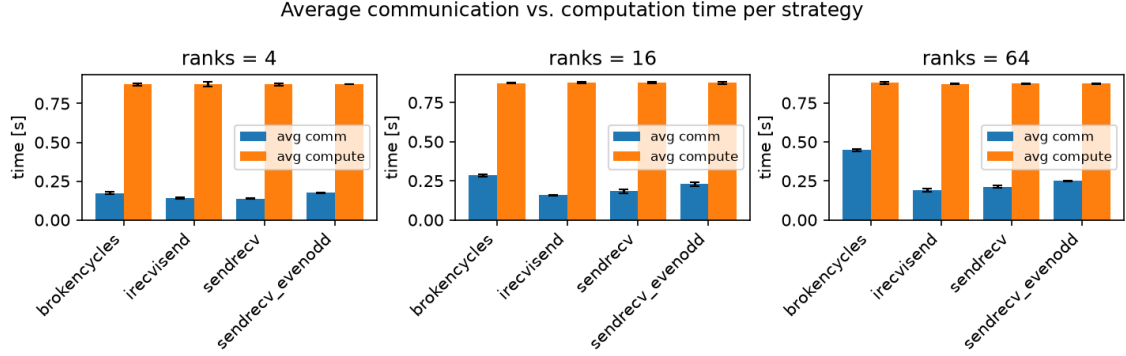


Figure 3: Average communication vs. computation time per strategy, weak scaling (128×128 local domain per rank, $n_z = 64$, 4 ranks per node). We omit measuring the time to copy fields in between diffusion steps.

Figure 4 shows the total runtime (communication and computation) against the number of ranks. The three strategies `irecvsend`, `sendrecv`, and `sendrecv_evenodd` show similar close to flat weak scaling behavior, where `sendrecv_evenodd` has larger initial overhead compared to the library solutions. The reference `irecvsend` seems to be the fastest strategy overall, but further testing would be necessary to see if for some settings a blocking communication pattern performs better. The `brokencycles` strategy shows clearly worse scaling behavior as predicted in Section 2, since its communication scheme becomes more expensive as more ranks are involved and the dependency chain grows.

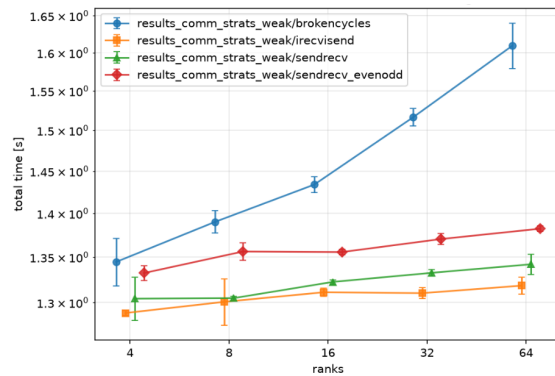


Figure 4: Mean time per rank vs. rank count, weak scaling. Three strategies stay close to flat, `brokencycles` climbs steadily.

Figure 5 turns this into a score per rank count. Parallel efficiency is the runtime at the smallest rank count divided by the runtime at each larger rank count, so a value of 1.0 means perfect weak scaling, indicated in the plot by the dashed 'ideal' line. This shows again that `irecvsend`, `sendrecv`, and `sendrecv_evenodd` all stay close to the ideal line across the range tested and that `brokencycles` instead drops steadily.

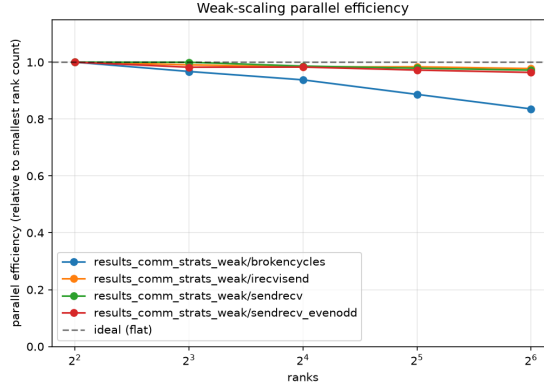


Figure 5: Weak-scaling parallel efficiency relative to the smallest rank count (4 ranks).

A plausible explanation for why **brokencycles** degrades this way is the shape of its communication pattern as indicated in Section 2.1. Rather than every neighboring pair of ranks exchanging data independently (as the other three strategies effectively do), **brokencycles** passes the data around a ring of ranks. The first rank in the ring waits until all others have completed their communication before finally receiving its data. With more ranks joining the ring, the dependency chain grows as does the wait time of the ranks, degrading the efficiency.

A similar idea also explains the ordering we see among the three deadlock-free strategies, not just why **brokencycles** falls behind. **sendrecv_evenodd** avoids deadlock using fully blocking **MPI_Send** and **MPI_Recv** calls, issued one at a time. Each call only returns once that single message has completely finished, so the four halo messages exchanged per phase are resolved strictly in sequence, even though the underlying network is capable of handling several messages at once. **sendrecv** does better because a single **MPI_Sendrecv** call bundles one send and one receive together, letting the library manage that pair internally rather than forcing the program to wait for one to finish before starting the other. This halves the number of sequential waiting points compared to **sendrecv_evenodd**. **irecvsend** goes further still: its non-blocking calls return immediately after handing a message to the network, so all four messages of a phase can genuinely overlap in hardware, and the program only waits once, for however long the slowest of the four takes, rather than for the sum of all four. This is consistent with the ordering we observe throughout Section 3.1.

3.1.2 Strong Scaling

In a strong-scaling experiment, the total problem size is held fixed and the work is split across a growing number of ranks. Each rank’s share of the problem shrinks as rank count grows. The goal is for runtime to drop in proportion to the number of ranks used.

In line with the results of Section 3.1.1, Figure 6 shows that **brokencycles** has the worst strong scaling behavior. The other three show similar scaling behavior with **irecvsend** seeming to perform slightly better.

Figure 7 shows a more interesting aspect of the strong scaling experiment. All four strategies show a superlinear speedup, that is, doubling the ranks results in a more than double speedup. As the problem is divided into smaller and smaller pieces per rank, each rank’s slice can eventually become small enough to fit into the processor’s cache instead of the slower main memory. This gives every rank a computation speedup on top of the benefit of simply splitting the work and more than compensating for the cost of additional communication. Note, however, that this can be called a free speedup due to hardware effects and has nothing to do with the communication strategies tested.

3.2 Overlapping Computation and Communication

For one rank on one node with a $128 \times 128 \times 64$ grid, the measured execution time consisted of approximately 30% communication and 70% computation. The resulting upper bound for speedup is therefore approximately $1.43\times$.

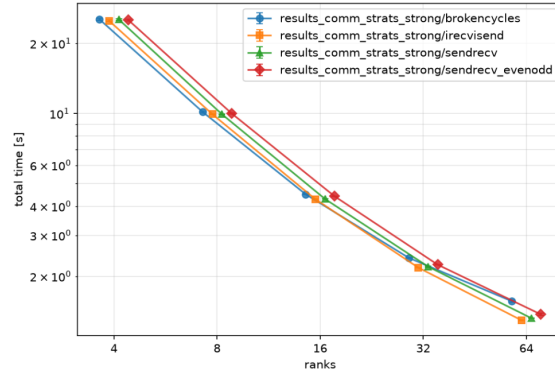


Figure 6: Mean time per rank vs. rank count, strong scaling (1024×1024 domain, $n_z = 64$).

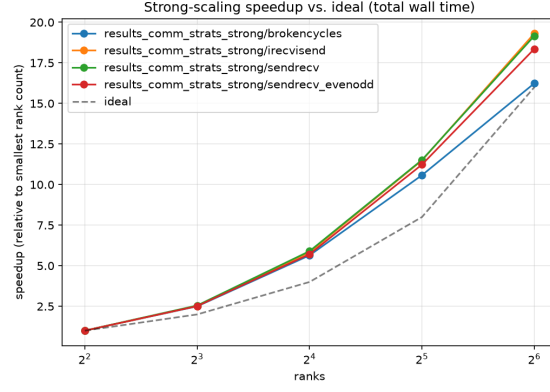


Figure 7: Strong-scaling speedup relative to the smallest rank count (4 ranks), vs. ideal linear speedup.

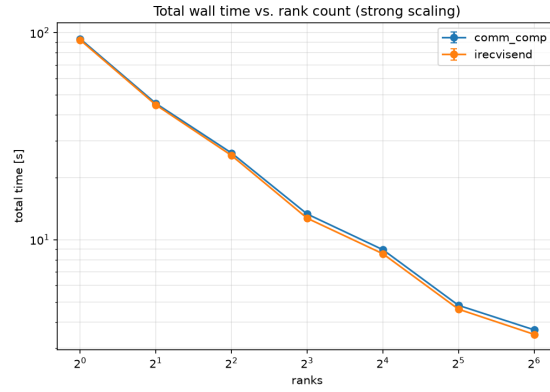


Figure 8: Strong-scaling comparison of the computation/communication overlap implementation and the non-blocking `irecvsend` reference on a fixed $1024 \times 1024 \times 64$ domain.

This bound shrinks as the workload becomes more distributed. For eight ranks on eight nodes with a $512 \times 1024 \times 64$ grid, measured communication and computation times were approximately 0.3 s and 3.2 s, giving an ideal upper bound of about $1.09 \times$. The ratio remains similar even with several ranks per node, since the goal there is still to exploit the combined compute power of multiple nodes rather than to shrink the per-rank workload.

Figure 8 shows the strong-scaling comparison between `comm_comp` and the non-blocking `irecvsend` reference. The global domain is kept fixed while the rank count increases from 1 to 64 with 4 ranks per node. The two implementations exhibit similar scaling, but `comm_comp` does not outperform the reference at any tested rank count.

The weak-scaling experiments showed the same qualitative behavior: `comm_comp` reduced part of the communication wait but did not produce a consistent total-runtime improvement over `irecvsend`.

To confirm that computation and communication were actually overlapping in `comm_comp`, we ran communication diagnostics with eight MPI ranks on eight nodes (one rank per node, $512 \times 1024 \times 64$ global domain, 1024 iterations). The original schedule spent approximately 0.16s in the measured `middle` wait; starting the left/right exchange before the interior computation reduced this wait to between approximately 0.02 and 0.12s.

3.3 Explicit Corner Exchange

We benchmark the three explicit-corner strategies against the non-blocking `mpi` baseline on Santis, scaling from 8 to 96 MPI ranks under strong and weak scaling, with identical rank placement throughout. We start at 8 ranks because Santis runs four ranks per node, so every configuration spans at least two nodes and exercises inter-node communication.

Figures 9 and 10 compare the mean time spent in `update_halo` under strong and weak scaling, respectively.

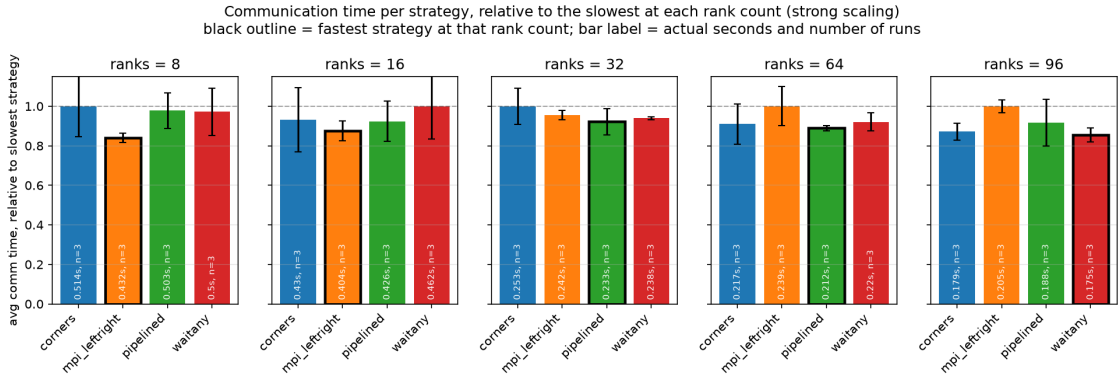


Figure 9: Corner-strategy halo-exchange time under strong scaling.

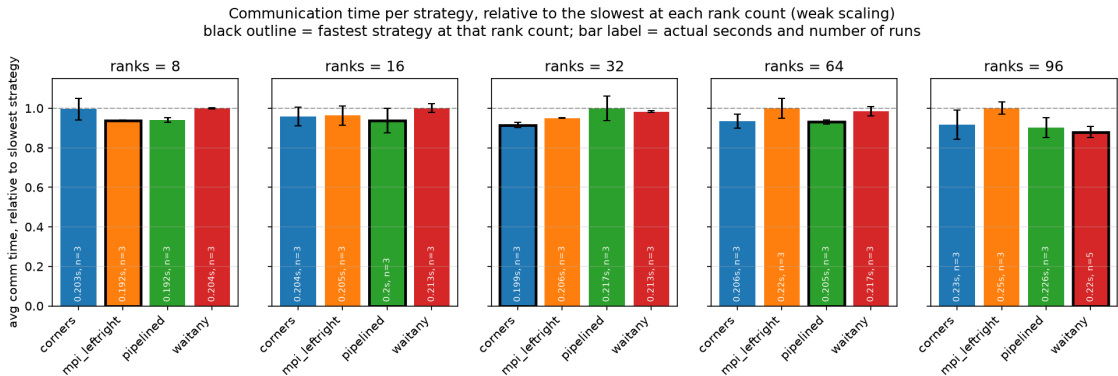


Figure 10: Corner-strategy halo-exchange time under weak scaling.

At high rank counts, all explicit-corner strategies outperform the non-blocking baseline. The standard **corners** strategy improves on the baseline by removing the dependency between its two communication phases. The optimized **pipelined** strategy is fastest at 64 ranks, while **waitany** is fastest at 96 ranks. Thus, once communication spans many nodes, avoiding waits can outweigh the cost of sending four additional corner messages. In general we focus on the behavior for a large number of ranks and we assume that the trends in speedup for increased numbers of ranks continue.

The detailed timers separate packing and unpacking, posting MPI calls, and waiting for transfers (Appendix A). Figures 11 and 12 show that packing time is similar for most strategies. It is higher for **waitany** because the code identifies and unpacks each completed message separately after `MPI.Waitany`, rather than unpacking all messages together after one `MPI.Waitall`.

The absolute-time labels in the appendix plots show that packing and unpacking together with MPI waiting account for most of the halo-update time; posting sends and receives is a much smaller component. A large relative difference in the send timer therefore has only a small effect on the total time.

Figures 13 and 14 show that the **pipelined** send timer is roughly twice as large as for the other explicit-corner strategies. This timer includes three separate posting groups and two `MPI.Testall` calls, after the top/bottom and left/right groups. The calls let MPI advance messages that have already been posted while the program prepares the remaining buffers. Their overhead is counted as send time. In the standard strategy, data transfer starts only after all buffers have been packed and all sends posted; in **pipelined**, the top/bottom transfers can proceed while the left/right and corner buffers are still being packed. Consequently, fewer transfers remain unfinished when the code reaches `MPI.Waitall`, reducing the time spent waiting (Figures 15 and 16). At 64 ranks this saving is larger than the extra posting overhead, giving **pipelined** the lowest overall halo time.

In Figures 15 and 16, **waitany** spends the least time waiting at 96 ranks. It unpacks a message as soon as that message arrives while other transfers continue. At lower rank counts, the bookkeeping for handling messages one at a time costs more than the overlap saves; at 96 ranks, the reduction in waiting is large enough to make **waitany** fastest with an overall communication time reduction of about 15 percent compared to the baseline approach.

The same mechanism explains both weak and strong scaling. As rank count grows, more halo traffic crosses between nodes and arrives less uniformly, creating slack between the first and last messages. **pipelined** exploits this by starting transfers as soon as they are packed, and **waitany** by unpacking each message as it arrives. Both benefit more as arrival times grow more uneven, which is why their advantage grows with rank count. At low rank counts, neighbors are mostly on the same node and messages arrive together, so there is little slack to exploit and the overhead of extra MPI calls and bookkeeping outweighs the benefit of these strategies.

4 Discussion and Conclusion

Several new communication strategies for the halo exchange step of a 4th-order diffusion stencil were successfully implemented, but not all lead to a reduction of overall runtime. For many strategies, our communication models were too simplistic: the expected speedups either did not materialize or appeared only at large rank counts.

As shown in Section 3.1.1, the ranking among the deadlock-avoidance strategies tracks how much sequential waiting each one builds into its message order, from `sendrecv_evenodd` (one wait per message) to `sendrecv` (one wait per pair) to the non-blocking `irecvsend`, which was fastest overall. The same principle explains why **brokencycles** scales worst: its ring-based dependency chain adds no extra data or messages, only extra sequential waiting that grows with rank count.

Two further approaches target the synchronization overhead through different mechanisms: **comm.comp** overlaps it with computation, while **corners** removes it outright. Neither reduces the total volume of data exchanged between ranks. **corners** sends four additional messages per exchange. **comm.comp** instead adds communication bookkeeping and splits the Laplacian computation into an interior region (computable while communication is in flight) and a boundary region (which must wait for the halo data), along with the associated request management and synchronization. Additional timers confirmed that this revised schedule successfully overlaps communication with computation,

but the reduction was too small to offset the added overhead: communication wait dropped without any corresponding gain in total application runtime. Further experiments would be needed to understand the exact sources of this overhead.

corners, by contrast, removes a synchronization point entirely. In the baseline strategies, the left/right exchange cannot start until the top/bottom exchange has completed on *every* neighboring rank, so a rank that finishes early still has to wait for its slowest neighbor before posting the next phase. **corners** eliminates this dependency by posting all eight edge and corner exchanges together in a single phase. The four additional messages are a small fixed cost, whereas the removed delay grows with rank count: for strong scaling at 64 ranks **pipelined** wins (about 11% faster communication), and at 96 ranks **waitany** wins (about 15% faster communication).

Our experiments suggest that minimizing waiting time is more critical than reducing message count. However, overlapping communication and computation shows that interactions are complex. Isolating and quantifying the relative impact of sequential delays versus message volume will require further testing.

Overall, the best strategy depends on domain size and rank count: our experiments favor a non-blocking **irecvisend** strategy, with **waitany** or **pipelined** becoming best only at high rank counts, while **comm_comp** could become beneficial on systems or workloads with a larger communication fraction and sufficient independent computation to hide it. However, no such benefit was observed in the configurations tested here.

5 Outlook

Several directions could naturally extend this work:

- Further investigation of the overhead limiting the performance of the **comm_comp** implementation and design of an adversarial setup, where one can deliberately complete the interior computation during the halo communication.
- Try the communication strategies for more complex stencils, for example including vertical communication needs.
- Further test higher rank counts to verify if the observed trends continue and isolate the effect of intra-node communication.

A Detailed Corner-Exchange Timings

A.1 Packing and Unpacking

Figures 11 and 12 show the packing timer used in the discussion of Section 3.3. The timer includes outgoing-buffer packing and received-buffer unpacking; for `waitany`, it also includes selecting and processing each individually completed request.

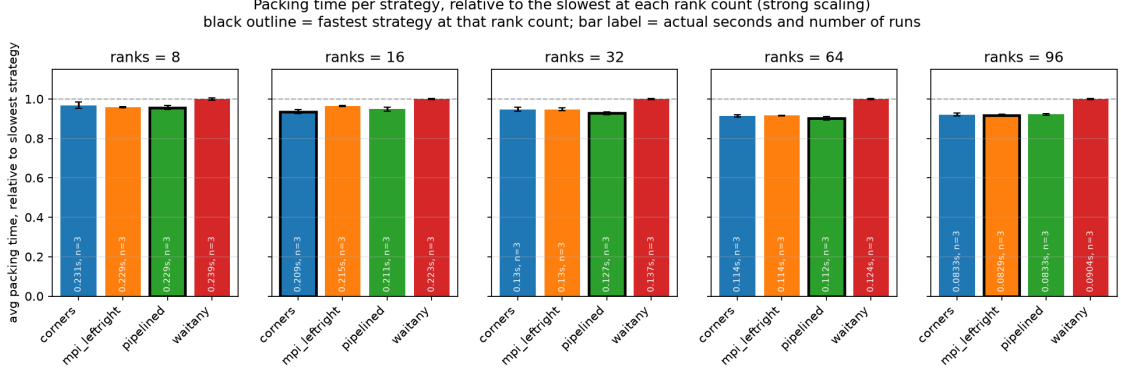


Figure 11: Packing and unpacking time by strategy under strong scaling, normalized to the slowest strategy at each rank count.

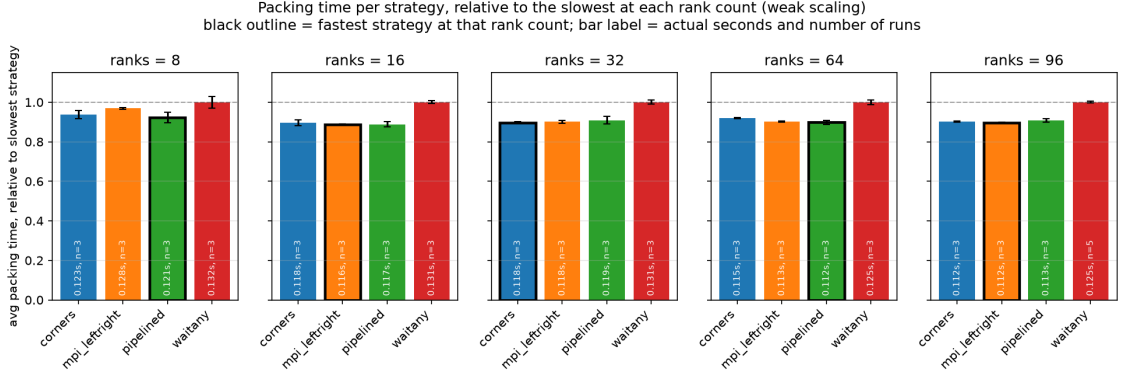


Figure 12: Packing and unpacking time by strategy under weak scaling, normalized to the slowest strategy at each rank count.

A.2 Send and Receive Initiation

Figures 13 and 14 show the time attributed to posting non-blocking sends and receives. For the pipelined variant this timer additionally contains the intermediate `MPI_Testall` progress calls.

A.3 Waiting for Communication

Figures 15 and 16 show the time spent inside `MPI_Waitall` or the repeated `MPI_Waitany` calls. This is exposed communication latency for which the rank cannot perform useful work.

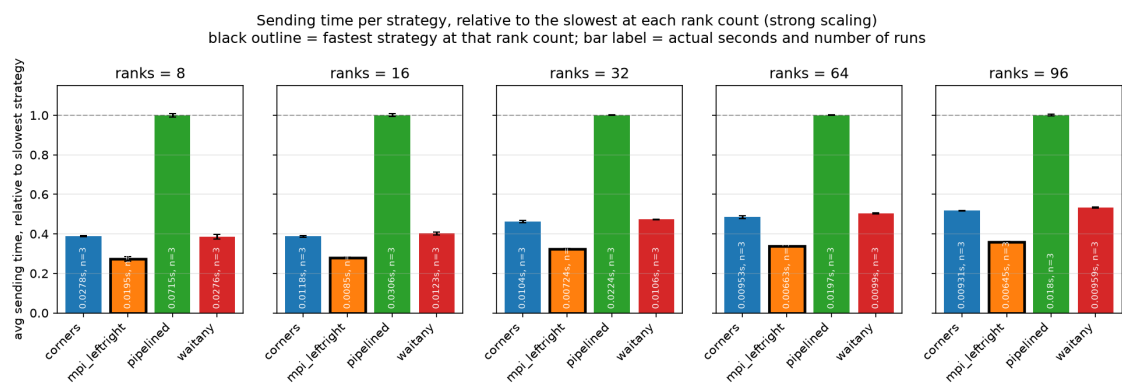


Figure 13: Send/receive initiation time by strategy under strong scaling, normalized to the slowest strategy at each rank count.

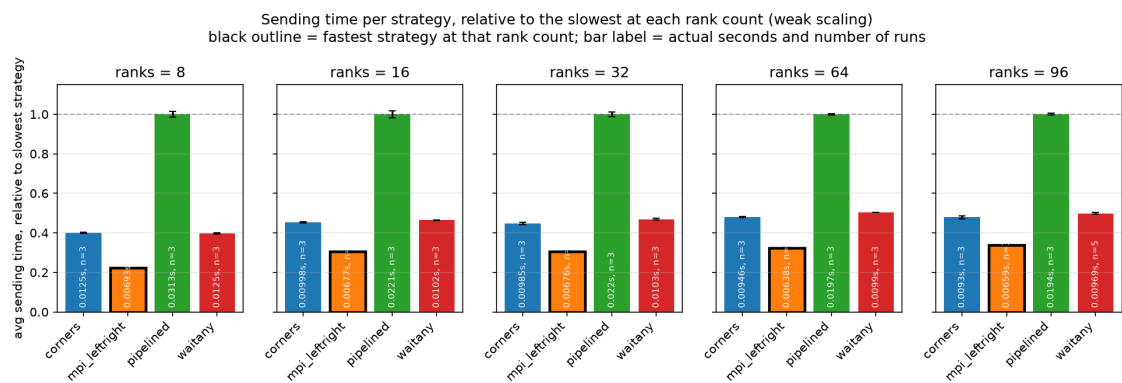


Figure 14: Send/receive initiation time by strategy under weak scaling, normalized to the slowest strategy at each rank count.

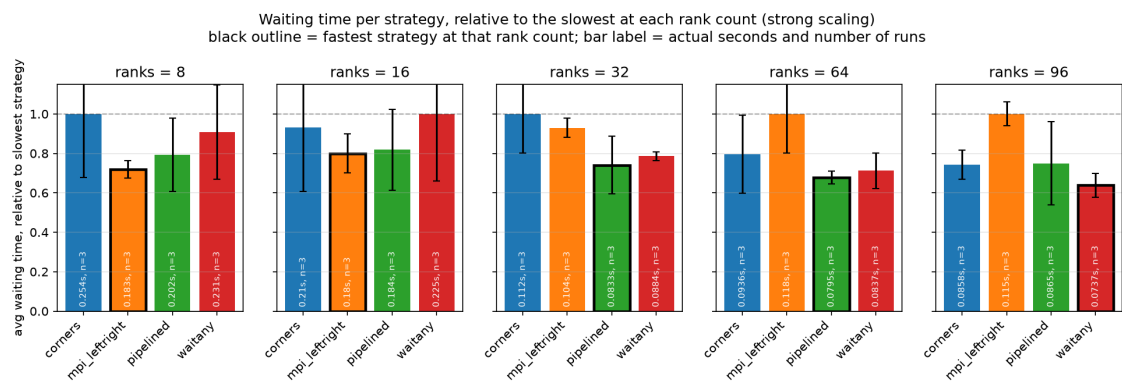


Figure 15: MPI waiting time by strategy under strong scaling, normalized to the slowest strategy at each rank count.

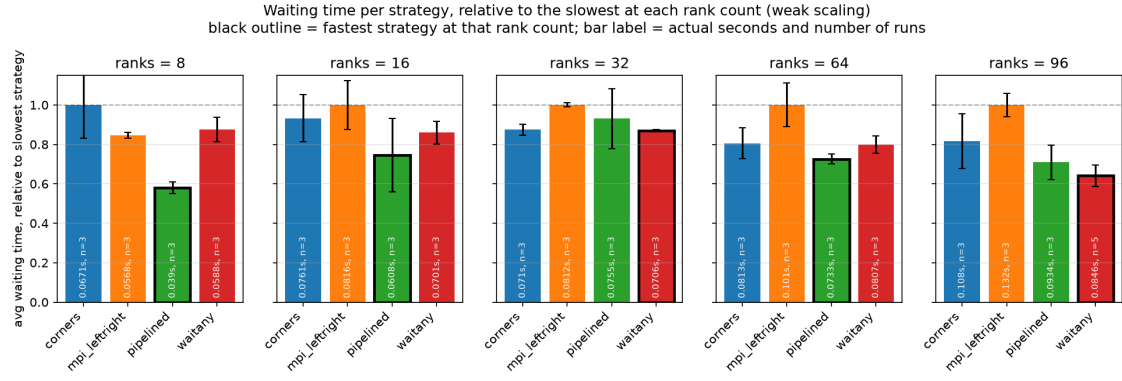


Figure 16: MPI waiting time by strategy under weak scaling, normalized to the slowest strategy at each rank count.

B Statement of AI use

We used AI tools (LLMs) to help write code and this report. Ideas and content are our own.